

Trabajo Práctico

"SubastaYa" es una plataforma web de subastas en tiempo real y comercio electrónico que busca modernizar las compras y ventas competitivas en línea.

El modelo de negocio se apoya en dos pilares críticos:

1. **Confianza y Solvencia Económica:** En las plataformas tradicionales, los usuarios suelen ofertar sin tener fondos reales, generando transacciones canceladas y fricción con los vendedores. En SubastaYa, **toda puja debe estar respaldada por saldo real congelado en garantía (Escrow)** dentro de la billetera virtual del usuario.
2. **Juego Limpio (Fair Play en Subastas):** Muchos sistemas sufren de *sniping* (bots o usuarios que ofertan en el último milisegundo para ganar sin dar tiempo a respuesta). Para evitarlo, la plataforma incorpora un mecanismo de **extensión dinámica de tiempo (Anti-sniping)** y una interfaz altamente reactiva donde el usuario tiene visibilidad total del estado de las ofertas en vivo.

El objetivo del equipo de desarrollo es construir una solución web completa (**Frontend, Backend y Base de Datos**) que ofrezca una experiencia visual atractiva y fluida para compradores y vendedores, respaldada por un motor transaccional seguro y confiable.

1. Requerimientos Funcionales y Experiencia de Usuario (Frontend & Flujo)

La aplicación web debe contar con una interfaz responsiva, intuitiva y visualmente cuidada. Se espera que los alumnos desarrollen una experiencia de usuario interactiva dividida en los siguientes módulos:

Módulo 1: Catálogo y Exploración de Subastas

- **Filtros y Búsqueda:** El usuario debe poder explorar subastas filtrando por:
 - **Estado:** *Activas* (en curso), *Próximas* (programadas para iniciar a futuro) y *Finalizadas*.
 - **Categoría:** Ej. Tecnología, Coleccionables, Vehículos, Arte, etc.
 - **Rango de precios y ordenamiento** (por menor tiempo restante o mayor puja).

- **Cards de Producto Informativas:** Cada tarjeta en el catálogo debe mostrar imagen referencial, título, categoría, oferta más alta actual, cantidad de ofertas realizadas y un contador regresivo visible.

Módulo 2: Creación y Publicación de Subastas (Vendedor)

- **Formulario Interactivo de Publicación:**
 - Datos del producto: Título, descripción detallada, URL de imagen y categoría.
 - Configuración económica: Precio base inicial y monto mínimo de incremento por puja.
 - Ventana temporal: Fecha/hora de inicio y fecha/hora de finalización.
- **Validaciones en pantalla:** La fecha de finalización debe ser posterior a la de inicio; el incremento mínimo y el precio base deben ser valores positivos coherentes.

Módulo 3: Sala de Subasta en Vivo (Live Bidding Room)

Es la vista principal de interacción en tiempo real:

- **Temporizador en Vivo:** Reloj visual con cuenta regresiva. Si el tiempo entra en zona crítica (último minuto), debe cambiar de color (ej. amarillo/rojo) para alertar a los postores.
- **Historial de Ofertas en Pantalla:** Lista cronológica con las últimas pujas (monto, usuario anonimizado/seudónimo y hora exacta de la oferta).
- **Consola de Puja Dinámica:**
 - Sugerencia automática del próximo valor a ofertar: $\text{\$}\{\text{Puja Actual}\} + \text{\$}\{\text{Incremento Mínimo}\}$.
 - Posibilidad de ingresar un monto personalizado superior.
 - Indicador visual inmediato de estado:
 - *Liderando:* Si la oferta más alta pertenece al usuario actual.
 - *Superado (Outbid):* Si otro usuario acaba de enviar una oferta superior.
- **Feedback y Alertas:** Mensajes visuales claros (*toasts* / modales) cuando se confirma una puja, cuando los fondos son insuficientes o si el tiempo de la subasta fue extendido por regla anti-sniping.

Módulo 4: Billetera Virtual y Gestión de Fondos

- **Panel de Saldo:** Visualización clara de tres métricas financieras:
 1. **Saldo Total:** Fondos totales depositados en la cuenta.
 2. **Saldo Retenido / En Garantía:** Monto actualmente bloqueado en subastas activas donde el usuario es el postor líder.
 3. **Saldo Disponible:** $\text{\$}\{\text{Saldo Total}\} - \text{\$}\{\text{Saldo Retenido}\}$ (único dinero disponible para nuevas pujas o retiros).
- **Carga de Saldo Simulada:** Formulario para acreditar saldo ficticio a la billetera.
- **Historial de Movimientos:** Tabla con el detalle de ingresos, retenciones por ofertas, liberaciones por superación y débitos finales por subastas ganadas.

Módulo 5: Panel de Usuario ("Mis Actividades")

- **Pestaña "Mis Compras / Pujas":** Listado de subastas donde participó, indicando si ganó el producto o si la subasta sigue abierta.
- **Pestaña "Mis Publicaciones":** Subastas creadas por el vendedor, con métricas de recaudación y estado de adjudicación.

2. Lógica de Dominio y Reglas del Negocio (Backend & Procesos)

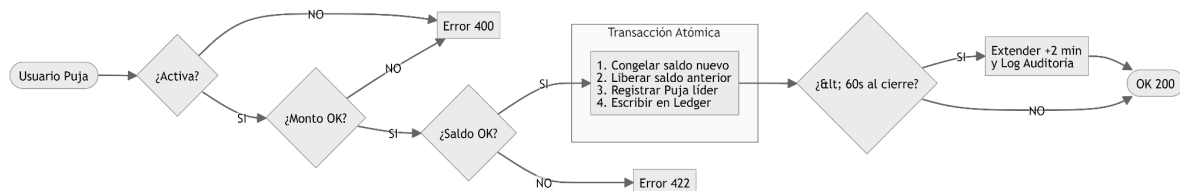
El backend debe garantizar la integridad de las transacciones y la correcta aplicación de las reglas del negocio:

2.1 Manejo Atómico de Saldos (Garantía / Escrow)

- Cuando un usuario realiza la puja más alta, su saldo disponible pasa a estar **retenido** por el valor de su oferta.
- Si un segundo usuario supera esa oferta válidamente:
 - El sistema debe **liberar de forma automática e inmediata** la retención del primer usuario, devolviéndole su saldo disponible.
 - Se congela el saldo correspondiente al segundo usuario.
- Ambas operaciones (liberación del anterior y bloqueo del nuevo) deben ejecutarse en un **bloque transaccional atómico**.

2.2 Regla Anti-Sniping (Extensión Automática de Tiempo)

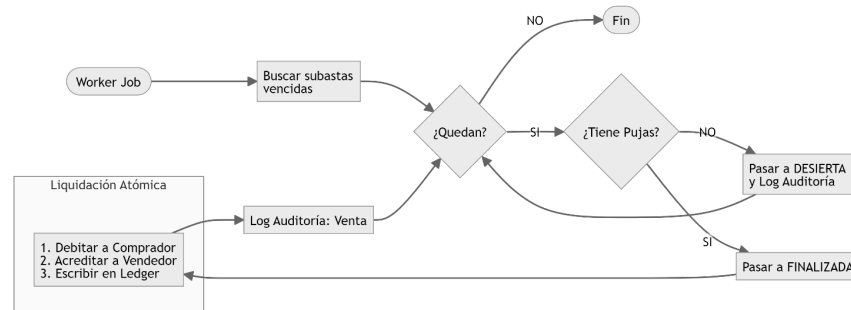
- Si una oferta válida se registra dentro de los últimos **60 segundos** previos al cierre de la subasta, el sistema debe extender automáticamente la fecha de finalización por **2 minutos adicionales**.
- Esto permite que los demás participantes tengan tiempo de reaccionar y evita victorias injustas por latencia de red.



2.3 Adjudicación y Procesos en Segundo Plano (Background Worker)

- Debe existir un proceso en segundo plano (Worker / Tarea programada / Cron job) que verifique periódicamente las subastas cuyo tiempo haya finalizado:

- **Con ganador:** Marca la subasta como **FINALIZADA**, transfiere el saldo retenido de la billetera del comprador a la del vendedor (liquidación final) y registra la venta.
- **Sin ofertas:** Si vence sin ninguna puja registrada, pasa a estado **DESIERTA**.



3.4 Auditoría de Eventos y Trazabilidad (Audit Log) Más allá del registro contable (*Ledger*) y el historial de pujas, el sistema debe registrar de forma inmutable los eventos críticos del negocio y del sistema en una tabla de auditoría. Se deben auditar obligatoriamente:

- Cambios de estado de las subastas (ej. paso de **ACTIVA** a **FINALIZADA** ejecutado por el *Worker*).
- Extensiones de tiempo gatilladas por la regla Anti-Sniping.
- Intentos de puja rechazados por concurrencia o validaciones de negocio críticas.
- Acreditaciones manuales de saldo en las billeteras.

3. Lineamientos y Restricciones Técnicas

El equipo asumirá el rol de **Arquitectos de Software**. No se proveerá un diagrama de base de datos ni un archivo OpenAPI predefinido. El equipo debe diseñar estas estructuras y justificar sus decisiones basándose en los siguientes estándares de la industria:

3.1 Backend y Base de Datos

- **Stack Tecnológico:** Arquitectura API REST utilizando alguno de:
 - C# (EF Core)
 - Java (Spring/Hibernate),
 - Python (SQLAlchemy),
 - Go (GORM)
 - Node.js (Sequelize/TypeORM).
- **Code-First:** El esquema relacional debe generarse obligatoriamente mediante migraciones en el código.
- **Preparación para Concurrencia:** Es mandatorio incorporar mecanismos en el modelo (ej. un campo **Version**) para soportar *Optimistic Locking*.

- **Transaccionalidad (ACID):** Las operaciones críticas deben ejecutarse bajo un modelo transaccional. Ante cualquier fallo en la actualización de registros, estados o logs de auditoría, se debe garantizar un Rollback completo para asegurar la integridad de los datos.
- **Estándares RESTful:** Diseño de endpoints basados exclusivamente en sustantivos plurales y jerarquías de recursos (ej. `GET /api/v1/recursos/{id}/subrecursos`). Se prohíbe el uso de verbos en la definición de las URLs. Asimismo, se debe garantizar el uso correcto de códigos de estado HTTP para reflejar el resultado de la operación y un manejo de errores preciso (ej. retornar `409 Conflict` ante problemas de concurrencia o conflictos de estado, en lugar de errores genéricos 500).
- **Documentación:** La API debe autogenerar su documentación utilizando OpenAPI (Swagger UI).
- **Comunicación en Tiempo Real (Sala de Subastas):** Para garantizar una experiencia fluida y justa en la vista de subasta en vivo, se espera que el temporizador y las nuevas ofertas se sincronicen utilizando **WebSockets** (ej. SignalR en .NET, Socket.io en Node, Spring WebSockets). Como alternativa mínima aceptable, se permitirá la técnica de *Short-Polling* (consultas periódicas asíncronas desde el frontend cada 2-3 segundos), aunque la implementación exitosa de WebSockets será un fuerte diferencial en la nota de arquitectura.

3.2 Diagrama de base de datos sugerido

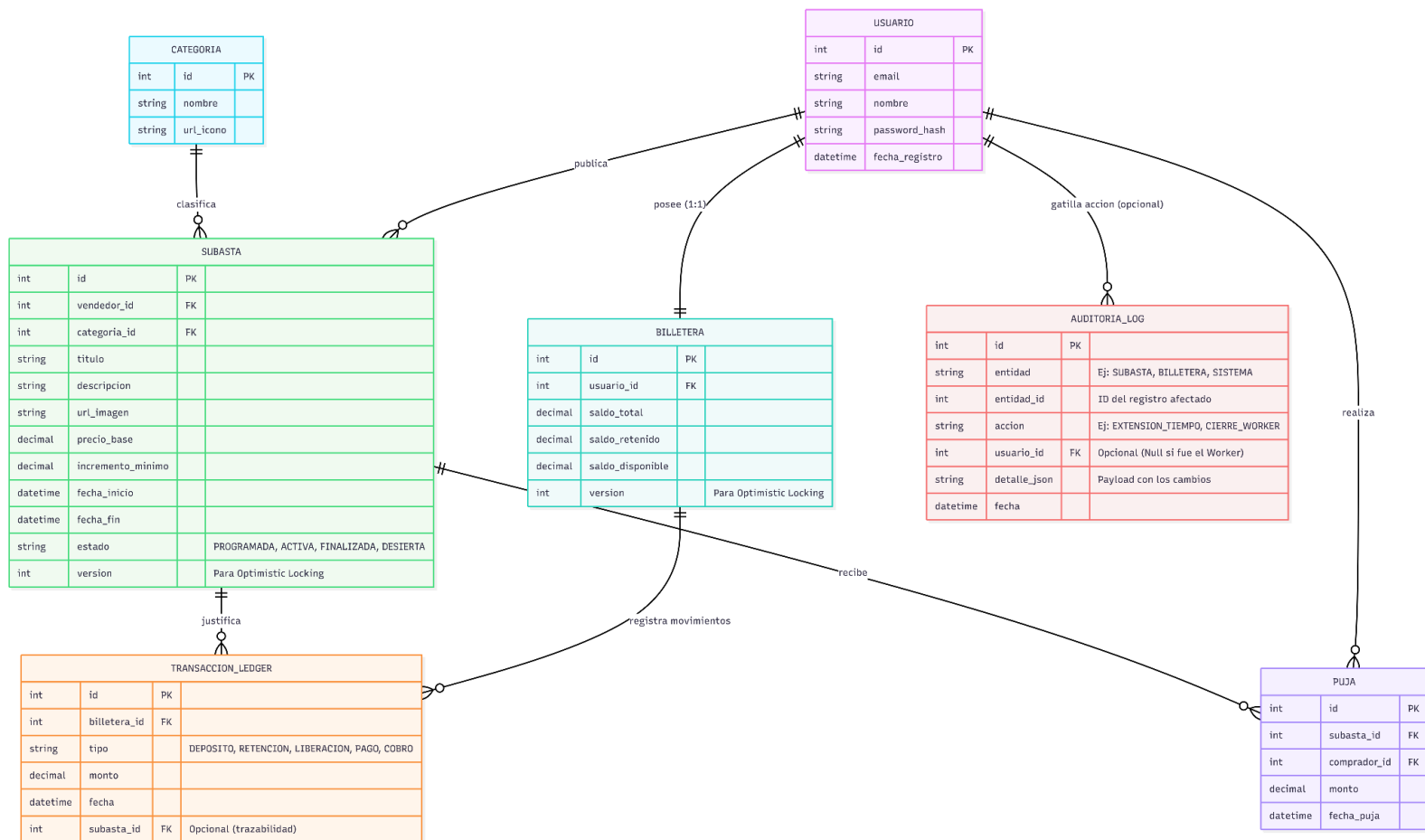


Diagrama *sugerido* de base de datos

3.3 Datos Semilla Obligatorios (Seed Data)

- **4 Usuarios y Billeteras:**
 - **vendedor@test.com:** Creador de publicaciones (Saldo \$0).
 - **comprador1@test.com:** Postor líder (Total: \$150.000 / Retenido: \$45.000 / Disp: \$105.000).
 - **comprador2@test.com:** Postor habilitado (Total: \$200.000 / Disp: \$200.000).
 - **sinfondos@test.com:** Sin saldo (Total: \$500, para probar rechazo de puja).
- **4 Categorías:**
 - Tecnología, Coleccionables, Indumentaria, Vehículos.
- **5 Subastas (Casos de Prueba):**
 - **Activa estándar:** Cierra en 20-30 min (con 2 pujas previas cargadas; líder \$45.000).
 - **Activa crítica:** Cierra en menos de 2 min (para probar alerta visual y extensión anti-sniping).
 - **Próxima:** Inicio programado a +24 hs (pujas bloqueadas).

- **Vencida con ganador:** Fecha fin pasada + puja ganadora (para probar cierre y liquidación del worker).
- **Vencida desierta:** Fecha fin pasada sin pujas (para probar pase a **DESIERTA**).
- **Registros Contables y Pujas:**
 - Historial de las 2 ofertas previas en la subasta activa.
 - Transacciones en el libro mayor (Ledger) que respalden depósitos y el saldo retenido de \$45.000.

3.4 Diseño de la API REST

El equipo definirá la totalidad de las rutas, parámetros y DTOs requeridos. A modo de **referencia de diseño**, el backend deberá contemplar endpoints similares a los siguientes (pudiendo adaptarlos o ampliarlos):

Recurso / Operación Sugerida	Propósito de Referencia
<i>GET /api/auctions</i>	Listado de subastas con paginación y filtros.
<i>POST /api/auctions</i>	Creación y parametrización de una nueva subasta.
<i>GET /api/auctions/{id}</i>	Detalle completo de la subasta, estado y puja actual.
<i>POST /api/auctions/{id}/bids</i>	Registro de una nueva oferta (evalúa saldo, incremento y anti-sniping).
<i>GET /api/wallet/balance</i>	Consulta del desglose de saldos (Total, Retenido, Disponible).
<i>POST /api/wallet/deposit</i>	Acreditación de fondos simulados en la billetera.

3.5 Frontend e Interfaz de Usuario (UI/UX)

- **Tecnologías:** Se permite el uso de HTML/CSS/JS (Vanilla) o frameworks modernos (React). Se recomienda el uso de librerías de estilos (Bootstrap, Tailwind, etc.) para asegurar un diseño limpio.
- **Integración:** El frontend debe consumir la API REST desarrollada en el backend de forma asíncrona (usando **Fetch API** o **Axios**).
- **Experiencia de Usuario (UX/UI):** Se requiere un diseño de interfaz de alta calidad, intuitivo y centrado en el usuario. La solución debe priorizar la claridad visual y la consistencia operativa, garantizando una retroalimentación constante y precisa ante cada interacción. El sistema debe asegurar que el usuario tenga plena visibilidad del estado de los procesos, proporcionando confirmaciones claras y una navegación fluida en todo momento.
 - EJ: Mostrar *spinners* o indicadores de carga mientras el backend responde.
 - EJ: Mostrar alertas claras de éxito o error
- **Validaciones:** Se debe prevenir el envío de peticiones innecesarias al backend (ej. deshabilitar el botón de un asiento que ya se muestra como ocupado).

3.6 Calidad de Código general

- **Clean Code:** Nomenclatura descriptiva, funciones pequeñas con responsabilidad única y comentarios enfocados en el "por qué" y no en el "qué".
- **Arquitectura:** Separación clara de responsabilidades (Controladores, Servicios/Casos de Uso, Repositorios). Evitar lógica de negocio acoplada a las vistas o a los controladores HTTP.

4. Pautas de Entrega y Evaluación

4.1 Modalidad de Entrega

- Repositorio de Código: La entrega del proyecto (tanto de la Entrega 1 como de la Entrega 2) se realizará exclusivamente a través de un repositorio **público** en GitHub.
- El repositorio debe contener tanto el código del Backend como el del Frontend.
- Se debe incluir un archivo **README.md** detallado que explique los pasos necesarios para compilar el proyecto, levantar la base de datos, ejecutar las migraciones y lanzar la aplicación.
 - **Prueba de Concurrencia (Stress Test):** El documento debe incluir una explicación breve (puede ser un bloque de código, un script de bash con **curl**, o exportación de Postman/JMeter) que demuestre cómo el equipo testeó la concurrencia optimista. Se debe probar que si se envían dos peticiones de puja idénticas en el mismo milisegundo, la base de datos registra solo una y rechaza la otra (ej. retornando HTTP 409 Conflict).
- Se debe enviar el enlace del repositorio al docente a través del campus virtual antes de la fecha y hora de cierre estipulada. (Si el repositorio es privado, se debe invitar al docente como colaborador).

4.2 Criterios de Evaluación

El proyecto se evaluará sobre una nota máxima de 10 puntos, distribuidos en los siguientes ejes:

1. Arquitectura y Base de Datos [2 pts]:

- Correcta normalización e integridad del esquema relacional.
- Implementación exitosa del enfoque *Code-First*.
- Manejo adecuado de la concurrencia (*Optimistic Locking*) y transaccionalidad (ACID) en operaciones críticas.

2. Estándares API REST y Lógica de Negocio [3 pts]:

- Cumplimiento estricto de las convenciones RESTful (URLs, Verbos HTTP).
- Uso correcto de códigos de estado HTTP (ej. **409 Conflict**, **400 Bad Request**, **200 OK**).
- Resolución correcta de las reglas de negocio (procesos en segundo plano, validación de caducidad).

3. Calidad de Código (Clean Architecture & SOLID) [2 pts]:

- Separación clara de responsabilidades en el backend.
- Código limpio, legible y modular.

4. Frontend y UX [3 pts]:

- Interfaz intuitiva y funcional.
- Correcto manejo asíncrono de las peticiones a la API.
- Retroalimentación visual adecuada frente a errores (especialmente fallas de concurrencia) y estados de carga.

4.3 Auditoría de Trabajo y Calificación Individual (IMPORTANTE)

El desarrollo de software es un esfuerzo colaborativo, pero la evaluación y calificación final será estrictamente individual.

- **Análisis de Repositorio:** El docente auditará el historial de *commits*, los *Pull Requests* y la pestaña de *Insights/Contributors* del repositorio de GitHub para evaluar el nivel de actividad, la frecuencia de los aportes y la calidad del código subido por cada uno de los integrantes del equipo.
- **Defensa del Proyecto:** Tras la entrega, el equipo realizará una defensa oral del trabajo práctico. Durante esta instancia, el docente podrá realizar preguntas técnicas sobre cualquier segmento del código, decisiones de diseño de la base de datos o arquitectura de la API a cualquier miembro del equipo.
- **Notas Disparas:** Si se evidencia una participación desigual en el desarrollo del proyecto (reflejada en el repositorio) o si un integrante no logra justificar las decisiones de implementación durante la defensa oral, el docente asignará notas disparas a los miembros del equipo, pudiendo resultar en la aprobación de un alumno y la desaprobación de otro dentro del mismo grupo.

RECURSOS:

1. Tutorial API Net Core:
<https://docs.microsoft.com/en-us/aspnet/core/tutorials/first-web-api?view=aspnetcore-3.1&tabs=visual-studio>
2. Tutorial API Spring Boot:
<https://www.nigmacode.com/java/Crear-API-REST-con-Spring>
3. Tutorial API Flask: <https://flask-restful.readthedocs.io/en/latest/quickstart.html>
4. Tutorial Go:
<https://blog.devgenius.io/simple-rest-service-in-golang-with-openapi-spec-and-orm-a447b1086e21>
5. Tutorial Node JS:
<https://www.freecodecamp.org/news/how-to-build-explicit-apis-with-openapi>
6. Tutorial HTML : <https://www.w3schools.com/html/default.asp>
7. Tutorial CSS : <https://www.w3schools.com/css/default.asp>
8. Tutorial JS: <https://www.w3schools.com/js/default.asp>
9. Framework de UI:
 - a. Bootstrap: <https://getbootstrap.com/>
 - b. Foundation: <https://get.foundation/sites.html>
 - c. Pure: <https://purecss.io/>
 - d. Skeleton: <http://getskeleton.com/>